Ngăn xếp và hàng đợi

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 1: Cấu trúc dữ liệu tuyến tính.
- Các bài trong chương: Mảng và danh sách liên kết; Ngăn xếp và hàng đợi.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

Mục tiêu bài học

Mô phỏng LIFO, FIFO và chọn stack hoặc queue đúng tình huống.

Kiến thức nền

Stack là ngăn xếp theo nguyên tắc LIFO: phần tử vào sau được lấy trước. push thêm vào đỉnh, pop lấy khỏi đỉnh, peek xem đỉnh mà không xóa. Với triển khai phù hợp, các thao tác cơ bản có thể O(1). Gọi pop trên stack rỗng cần được xử lý theo quy ước của chương trình.

Queue là hàng đợi theo nguyên tắc FIFO: phần tử vào trước được lấy trước. enqueue thêm vào cuối, dequeue lấy ở đầu. Có thể dùng danh sách liên kết hoặc bộ đệm vòng để tránh dịch toàn bộ phần tử mỗi lần lấy đầu.

Stack hữu ích cho kiểm tra dấu ngoặc và quay lui; queue hữu ích cho xử lý công việc theo thứ tự đến và duyệt BFS. Cấu trúc mô tả quy tắc lấy dữ liệu, không đồng nghĩa với một kiểu lưu bộ nhớ duy nhất.

1. Giao diện trừu tượng trước triển khai

Stack có push, pop, peek, is_empty; queue có enqueue, dequeue, peek. Quy định rõ thao tác trên cấu trúc rỗng: báo lỗi, trả sentinel hoặc một kiểu kết quả riêng. Không dùng None làm dấu lỗi nếu None cũng là dữ liệu hợp lệ mà không có cách phân biệt.

Queue cài bằng list và pop(0) trong Python thường phải dịch phần tử. deque hỗ trợ thao tác ở hai đầu phù hợp hơn cho queue. Đây là lựa chọn triển khai, không thay đổi quy tắc FIFO.

2. Truy vết hai cấu trúc

```
Thao tác        Stack trả      Queue trả
Thêm A          -              -
Thêm B          -              -
```

```
Lấy một         B             A
Thêm C          -             -
Lấy một         C             B
```

Hãy tự ghi phần tử còn lại sau từng dòng. Mô phỏng trên giấy giúp phân biệt quy tắc trước khi viết mã.

3. Kiểm tra dấu ngoặc bằng stack

```
def balanced(text):
    pairs = {')': '(', ']': '[', '}': '{'}
    stack = []
    for char in text:
        if char in '([{':
            stack.append(char)
        elif char in pairs:
            if not stack or stack.pop() != pairs[char]:
                return False
    return not stack

assert balanced('([])')
assert not balanced('([)]')
assert not balanced('(')
```

Đây là ví dụ kiểm tra chuỗi dấu ngoặc, không phải bộ phân tích cú pháp Python: ngoặc trong chuỗi ký tự hoặc comment chưa được xử lý theo ngữ pháp ngôn ngữ.

4. Bài tập và lời giải

Thêm test chuỗi rỗng, dấu đóng ở đầu, ba loại ngoặc lồng nhau. Với n ký tự, mỗi ký tự xử lý một lần: thời gian O(n), bộ nhớ tối đa O(n). Đối với hàng đợi công việc, nêu cách ứng xử khi đầy và khi rỗng; không suy ra FIFO đồng nghĩa mọi công việc hoàn tất theo thứ tự nếu nhiều worker chạy đồng thời.

> **Bài tập vận dụng bổ sung**
>
> thực hiện push(A), push(B), pop(), push(C), pop() trên stack. Làm chuỗi tương tự với enqueue và dequeue rồi so sánh kết quả.

Câu hỏi tự kiểm tra

1. Stack tuân theo nguyên tắc nào?
2. Queue tuân theo nguyên tắc nào?
3. push(A), push(B), pop() trên stack trả về gì?
4. Thao tác peek trên stack thường làm gì?
5. Thuật toán BFS thường dùng cấu trúc nào?

Gợi ý đáp án và điểm cần nhớ

- LIFO. LIFO nghĩa là vào sau, ra trước.
- FIFO. FIFO nghĩa là vào trước, ra trước.
- B. B được đưa vào sau nên nằm trên đỉnh.
- Xem đỉnh mà không xóa. peek đọc phần tử trên đỉnh nhưng giữ nguyên stack.
- Queue. Queue giúp xử lý các đỉnh theo thứ tự khám phá từng lớp.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html